

Geo-Spatial and web technologies
INFO-H509

Urban Mobility Usecase Delay Visualization

Authors

Imad Sharof
Yawa Aziable
Hassan Noor UI
Murielle Djiowa Tchamakam
Antonios Sasaridis

Professor

Mahmoud Sakr
Gaspard Merten

Academic year 2025–2026

Abstract

1 | Introduction

This project forms part of the INFO-H-509 course entitled ‘Geospatial and Web Technology’. The **MobilityTwin.Brussels** platform aggregates mobility data for the Brussels-Capital Region in real time and historically, drawing from more than ten different sources (STIB, SNCB, Infrabel, TEC, De Lijn, Lime, Bolt, etc.), representing over **4 terabytes of data** collected since 2023.

From the proposed topics, we chose the theme ‘**Delay Visualisation**’, which focuses on the analysis of rail delays on the Belgian network managed by Infrabel. Belgium has one of the densest rail networks in Europe, which makes the analysis of its punctuality all the more relevant for users and network managers.

Daily rail punctuality reports offer limited analytical value when taken in isolation. By aggregating data over longer time horizons, it becomes possible to identify significant trends, such as chronically delayed trains, problematic stations or network segments, as well as recurring daily or weekly patterns. This project addresses that gap by posing four operational questions :

1. **Which stations and rail lines are systematically delayed ?**
2. **Does delay follow predictable hourly and weekly patterns ?**
3. **Do the same train numbers cause recurring delays ?**
4. **How do disruptions propagate through the day ?**

The audience for this work spans multiple stakeholders : Infrabel operations engineers looking for evidence-based maintenance priorities, urban mobility researchers at MobilityTwin.Brussels, and commuters seeking to understand why their route systematically underperforms.

This report presents our data pipeline, the metrics developed and the interactive visualisations

produced to explore these dynamics using the Infrabel data available on MobilityTwin.Brussels.

2 | Data and Sources

2.1 Main Source : MobilityTwin.Brussels API

The rail punctuality data is sourced from the MobilityTwin.Brussels platform, which aggregates and redistributes data published by Infrabel, the Belgian rail infrastructure manager. Access is via the following endpoint :

GET `https://api.mobilitytwin.brussels/infrabel/punctuality?timestamp=<iso>`

Each request returns all punctuality records for the day corresponding to the specified timestamp. Authentication is handled via a Bearer token passed as an environment variable.

2.2 Structure of a record

Each record corresponds to a train stop at a station and includes the following fields :

Field	Description
train_no	Train identification number
train_serv	Service type (IC, S, L, etc.)
ptcar_lg_nm_nl	Station name (in Dutch)
relation	Origin–destination relation
datdep	Scheduled departure date
planned_date_arr / planned_time_arr	Scheduled arrival time
real_date_arr / real_time_arr	Actual arrival time
delay_arr	Arrival delay (in seconds)
delay_dep	Departure delay (in seconds)
line_no_arr / line_no_dep	Railway line number
relation_direction	Direction of the relation

TABLE 2.1 – Description of the railway dataset fields

The granularity is therefore fixed : a single train generates as many records as there are stations

it serves. Over the period analysed (May 2026, 7 days), this represents more than 470,000 arrivals spread across 447 stations and 103 lines.

2.3 Additional geographical data

Two static GeoJSON files are used to enhance the map visualisation :

File	Content
operational_points.json	GPS coordinates of Belgian railway stations
line_sections.json	Geometry of railway line sections

TABLE 2.2 – Description of the dataset files

These files enable delay metrics to be overlaid onto an actual map of the Belgian rail network.

2.4 Known limitations of the data

Several limitations were identified during the project. Some were inherent to the data source, while others were resolved through dedicated processing steps.

2.4.1 Unresolved limitations

- **Station names in Dutch only** (ptcar_lg_nm_nl) : harmonisation with the French names was necessary for certain visualisations.
- **Delays expressed in seconds** : a conversion to minutes has been applied solely for the sake of readability.
- **Lack of real-time data** : the endpoint returns daily data rather than a continuous stream, which limits intra-day analyses to a reconstructed hourly granularity.
- **Possible missing data** : certain days or stations may have incomplete records, particularly in the event of major disruptions.

2.4.2 Limitations addressed in the pipeline

- **API instability** (rate limiting timeouts) : resolved via an exponential backoff retry mechanism (up to 4 attempts) and a local disk cache per day.

- **Station name divergence** between the API and GeoJSON files : resolved by systematic normalisation to uppercase before each join.
- **Separate date/time fields** : the `planned_date_arr` and `planned_time_arr` fields were merged into full `datetime` objects during cleaning.
- **Missing stop ordering** : the sequential order of stops within a journey is not provided by the API, reconstructed using the median planned arrival time per station.
- **Public holidays not flagged** : automatically detected using the **holidays** library with the Belgian calendar, and exposed as a filter in the dashboard.

3 | Data Pipeline

The pipeline moves raw API records through 3 successive stages before they reach the dashboard.

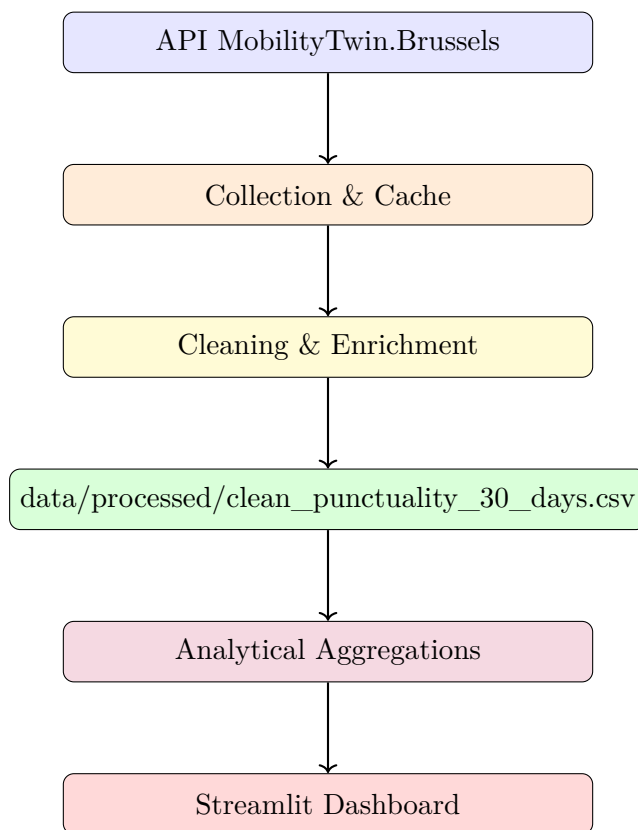


FIGURE 3.1 – General architecture of the data pipeline

3.1 Stage 1 — Fetching 30 Days of Data

The raw Infrabel dataset provides one row per individual train stop per operating day. Each row captures a single train passing through a single station on a specific trip, with its exact planned and actual arrival and departure times, and the resulting delay expressed in seconds. We collect 30 days of data to ensure that each station–hour combination is observed enough times to compute a meaningful average, and that the full weekly cycle is covered at least four times, making weekday patterns statistically visible. Data is fetched day by day from the MobilityTwin.Brussels API, with results cached so that repeated queries over the same period incur no additional network cost.

3.2 Stage 2 — Cleaning and Feature Engineering

The raw JSON is transformed into a structured analytical DataFrame through three operations. In the raw data, dates and times are stored as two separate strings (e.g. "2024-03-05" and "08 :32 :00"). The script concatenates them and parses them into proper datetime columns. Any value that cannot be parsed, due to missing or malformed data, is replaced by NaT (Not a Time), the standard pandas representation of a missing datetime.

To convert delays, we create human-readable minute equivalents by dividing by 60 the raw delay fields in seconds. We then derived from existing fields several new columns are : the hour of day (0–23) is extracted from the planned datetime, a train planned at 08 :12 gets hour 8, and so does a train at 08 :47 or 08 :59 ; the weekday name is extracted from the operating date ; a boolean weekend flag is added ; and binary flags mark whether the delay exceeds 5 minutes, the threshold used in Infrabel’s official punctuality statistics. Finally, only the analytically useful columns are kept, and the result is saved as a clean CSV.

3.3 Stage 3 — Adding Coordinates and Geometries

Since the raw punctuality data contains no geographic coordinates, it only has station names as text labels ; two separate scripts enrich the clean data with spatial information.

For stations, the script computes for each (station, hour) pair the mean arrival delay across all individual train stops that fall within that hour window over the 30 days. It then looks up the geographic coordinates of each station from a separate Infrabel GeoJSON file and joins them by

station name.

To observe the railway lines, the same aggregation is done per (line, hour) pair, and the delay values are joined to the line geometries. Because the geometry file contains over 1,000 individual track segments, they are first merged into approximately 260 unified line geometries before being joined with the delay aggregates. These two enriched GeoJSON files are the direct input to the interactive map.

4 | Technical Challenges

Four challenges shaped the implementation. First, neither GeoJSON file shared a numeric identifier with the CSV, so join keys were established empirically : station names were normalised to uppercase on both sides, and for lines the alphanumeric field `linecalfa` (e.g. "139/1") was selected over the integer `linecnum`, raising the match rate from roughly 7% to 79%.

Second, the 30-day punctuality dataset reaches approximately 450 MB ; memory pressure was controlled by passing `usecols` to `pd.read_csv`, loading only the columns required by each aggregation.

Third, all five filter dimensions — date range, weekdays, holiday exclusion, station, and relation — are applied as sequential pandas masks on already-fetched data, ensuring that sidebar interactions never trigger a new API call and responses remain sub-second.

Fourth, PyDeck's `PathLayer` requires coordinates as nested Python lists, so a conversion function extracts them from Shapely `LineString` and `MultiLineString` geometries produced by GeoPandas.

5 | Interface Implementation

To make the aggregated data accessible and interactive, we developed a web-based dashboard. The goal was to provide an intuitive tool allowing both technical and non-technical users to explore spatial and temporal delay patterns across the Belgian rail network without writing any code.

5.1 Technology Stack

The interface was built using Streamlit, an open-source Python framework designed for the rapid development of data applications. This choice allowed us to seamlessly integrate our existing Python data pipeline (relying heavily on pandas and GeoPandas) with interactive visual components, avoiding the overhead of managing a separate frontend architecture. Spatial rendering was handled through PyDeck and Plotly Express, providing high-performance, dynamic maps directly within the web application.

5.2 Dashboard Layout and Features

The dashboard is structurally divided into two main areas : a persistent control sidebar and a multi-tab visualization panel.

5.2.1 Interactive Sidebar

The sidebar acts as the central control panel for the application, capturing user inputs to filter the underlying data dynamically. It includes :

- **Period Selection** : Date pickers accompanied by quick presets (e.g., "Last 7 days", "This month") to define the historical analysis window.
- **Categorical Filters** : Multiselect dropdowns and toggles allowing users to narrow down the data by specific weekdays, public holidays, stations, or commercial rail relations (e.g., IC, S, L routes).
- **Temporal Navigation** : An interactive slider (0 to 23) enabling users to travel through the hours of the day to observe the state of the network at any specific time window.

5.2.2 Visualization Tabs

The main interface is organized into several tabs, each dedicated to a distinct analytical perspective :

- **Map** : Displays the geographic state of the network for the hour selected in the sidebar. Stations are represented by scatter points (sized by traffic volume) and track segments by paths. A dynamic color scale, ranging from green to red, instantly highlights areas suffering from severe average delays.

- **Patterns** : Features a weekday-by-hour heatmap to identify recurring structural delays across the weekly schedule, alongside a daily bar chart tracking the absolute count of significantly late arrivals over the chosen period.
- **Animation** : Provides a 24-hour animated playback of the map layer, effectively illustrating how disruptions initiate, propagate, and resolve over the course of a day.
- **Drill-down & Trains** : Allows users to isolate and inspect a specific station, train number, or route in detail, while automatically highlighting the worst-performing entities based on the selected filters.

5.3 Performance Optimization and Data Flow

Given the substantial volume of data—comprising hundreds of thousands of punctuality records and complex spatial geometries—performance optimization was a critical priority for the interface.

We implemented a robust caching strategy to ensure a fluid user experience. Heavy operations, such as loading the static GeoJSON coordinates for stations and dissolving the railway line geometries, are executed only once upon startup and stored in memory. Similarly, data fetched from the MobilityTwin API for a given date range is cached.

When a user interacts with the sidebar widgets (for instance, sliding the time selector from hour 8 to hour 9), the application does not trigger a full data reload. Instead, it applies instantaneous pandas filtering logic to the cached dataframes and immediately re-renders the PyDeck and Plotly layers. This decoupled architecture guarantees sub-second responsiveness during data exploration.

6 | Results and Analysis

The analysis covers 43 days of Infrabel punctuality data, from April 1 to May 13, 2026, representing tens of thousands of individual train stop records across the Belgian rail network.

6.1 Geographic distribution of delays

This first result of the visualization is a geographic map that reveals a clear spatial concentration of delays in the eastern part of Belgium. It allow us to identify the 10 worst stations at hour 16 : Beverlo, Beringen, Balen, Zonhoven, and Heusden, with mean arrival delays above 8 minutes, well beyond Infrabel’s 5-minute threshold. This clustering strongly suggests that the issue is not isolated to individual stations but reflects a structural problem on the rail lines serving the Limburg region.

24-hour propagation of delays across the network

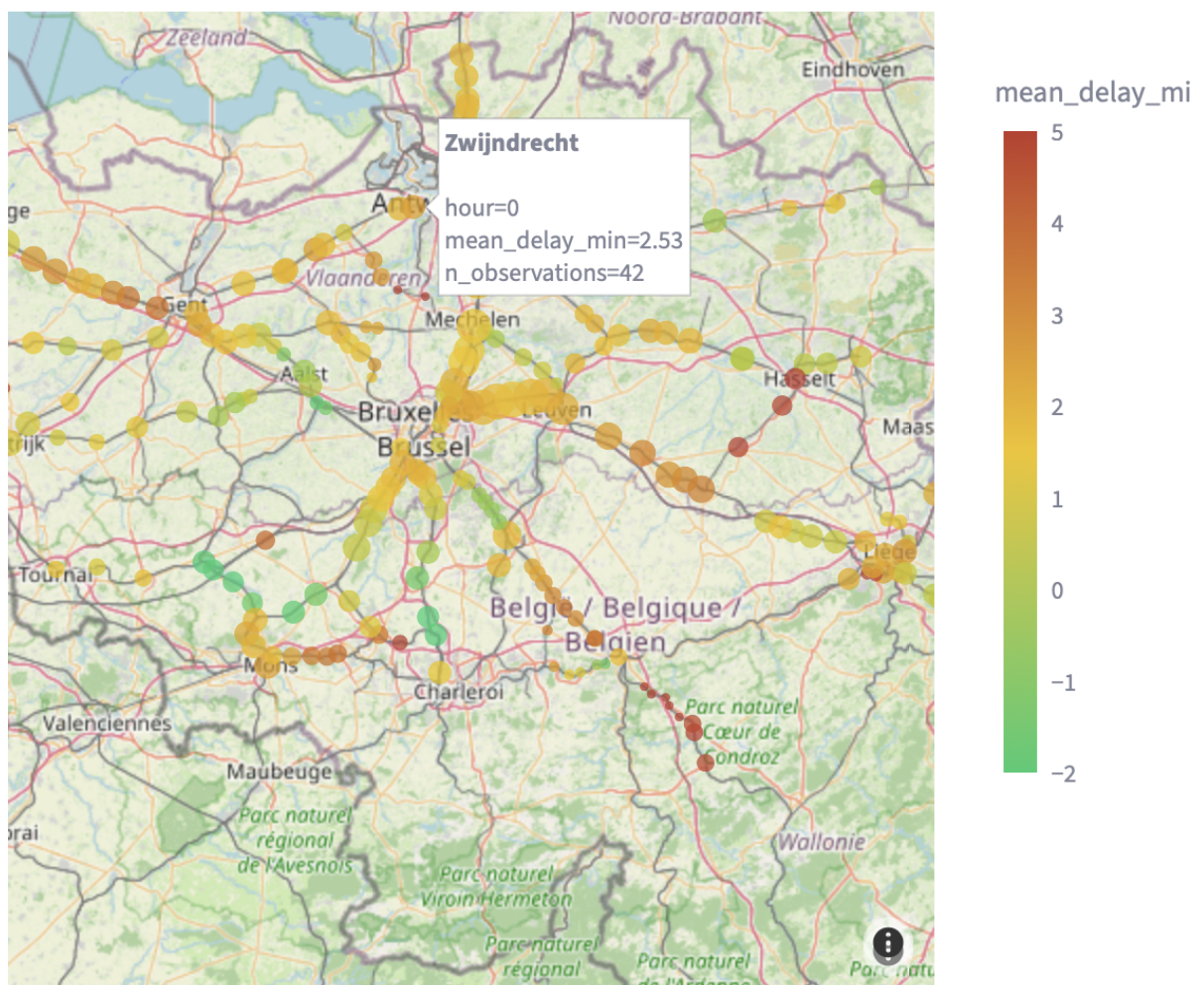


FIGURE 6.1 – Geographic distribution of delays

6.2 Delay pattern

From a daily point of view, we have the global mean delay curve that illustrates the delay pattern over the day and answers the question of whether those delays get resolved over the time.

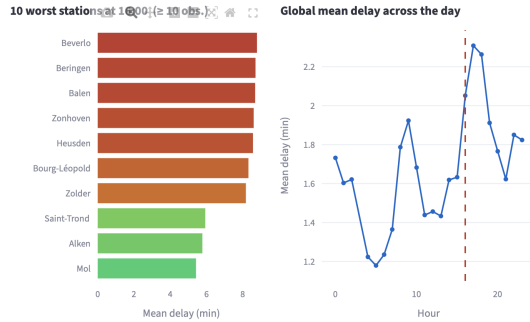


FIGURE 6.2 – Daily delay curve

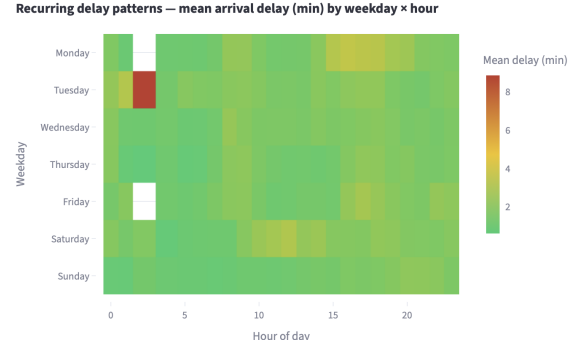


FIGURE 6.3 – Weekly heat map

A first moderate peak appears around hour 9, reaching approximately 1.9 minutes of mean delay. The most pronounced peak occurs around hour 17, where the mean delay climbs to approximately 2.3 minutes, the highest point of the entire day. This corresponds to the evening rush hour, when train frequency and passenger load are at their highest. Between these two peaks, delays drop notably around hours 4 to 5 (reaching their daily minimum near 1.2 minutes) and again around hours 11 to 13. After the evening peak, delays remain elevated through the late evening hours around 1.8 minutes, suggesting that disruptions accumulated during the peak do not fully resolve before the end of service.

Weekly speaking, the pattern is shown through a heatmap. In our case, we observe that the most severe delay episode in the entire 43-day period occurred on a Tuesday between hours 2 and 3, where the mean delay reaches above 8 minutes (in deep red). This is a striking outlier compared to the rest of the week, and likely reflects a specific incident on that night rather than a structural pattern, since no other Tuesday shows comparable values. More information can be retrieved from the heat map, like on Monday, when there are delays around hours 1 to 2.

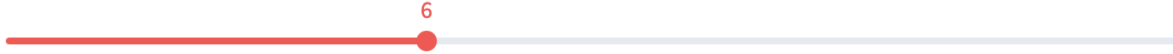
6.3 Recurring late trains

Other than that information we can also get information about the recurring late trains table, filtered at a threshold of 6 minutes over the 43-day period. This identifies the most structurally problematic train numbers, like train 475 (recorded 707 late arrival events out of 982 total stops

with a mean delay of 30.5 minutes and a maximum of nearly 119 minutes).

Trains whose arrivals are recurrently late on the selected period. Useful to flag timetable inputs that may need adjustment.

Late threshold (min)



Top N



	train_no	relation	late_events	total_stops	late_rate	mean_delay_min	max_delay_min
0	475	L	707	982	0.72	30.5	118.98
1	9115	EURST	551	590	0.934	28.69	64.2
2	9147	EURST	507	617	0.822	23.71	94.62
3	9157	EURST	482	781	0.617	22.69	171.52
4	2409	IC 18	475	1754	0.271	4.17	31.33

FIGURE 6.4 – Recurring late trains table

7 | Conclusion

This project successfully demonstrated the value of aggregating daily rail punctuality reports into a comprehensive, interactive analytical tool. By leveraging the MobilityTwin.Brussels platform, we developed a robust data pipeline capable of processing and enriching thousands of raw Infrabel records with spatial geometries. The resulting Streamlit dashboard effectively translates complex, multi-dimensional datasets into accessible geographic and temporal visualizations.

Our analysis answered the initial operational questions and revealed actionable insights. Ultimately, this work bridges the gap between raw open data and practical urban mobility insights. It provides a solid foundation that could be further expanded in the future for instance, by integrating real-time data streams, passenger load metrics, or weather conditions. Such an enhanced tool would continue to assist operators like Infrabel in prioritizing maintenance and optimizing schedules, while helping commuters better understand and navigate their daily journeys.